

■ 2.6.9 Character Strings

All input and output in *Mathematica* ultimately consists of strings of characters. There are some situations in which you may find it convenient to work directly with strings of characters. You can give any string of text in *Mathematica* in the form `"text"`.

You can put any text into a *Mathematica* string. The `"` are not displayed in standard *Mathematica* output format.

```
In[1]:= "This is a text string."
Out[1]= This is a text string.
```

The `"` are given in input form.

```
In[2]:= InputForm[%]
Out[2]//InputForm= "This is a text string."
```

The string `"x"` is not the same as the symbol `x`. In the standard output format, you do not see the `"` in the string `"x"`.

```
In[3]:= "x" != x
Out[3]= x != x
```

You can tell whether something is a string by looking at its head.

```
In[4]:= Head["x"]
Out[4]= String
```

You can define transformation rules that involve strings.

```
In[5]:= z["gold"] = 79
Out[5]= 79
```

Any standard printable character can appear in a *Mathematica* string. There are also ways to include special characters. You can use special characters to build up formats for special output devices.

<code>a</code>	an ordinary character
<code>\"</code>	a <code>"</code> to be included in a string
<code>\n</code>	a newline (line feed)
<code>\t</code>	a tab
<code>\nnn</code>	a character with octal code <i>nnn</i>

Special codes in *Mathematica* strings.

In *Mathematica*, a single character is represented as a string of length one. Most computer systems have a way of numbering characters, most often using ASCII codes. *Mathematica* can convert between strings, lists of characters, and ASCII codes.

<code>Characters["string"]</code>	give a list of the characters in a string
<code>ToASCII["c"]</code>	give the ASCII code for a character
<code>FromASCII[n]</code>	construct a character from an ASCII code

Conversions between strings, characters and ASCII codes.

Here is a string.

```
In[6]:= str = "The string."
Out[6]= The string.
```

This gives a list of the characters in `str`.
Each character is a string of length one.

```
In[7]:= Characters[str]
Out[7]= {T, h, e, , s, t, r, i, n, g, .}
```

Here is the fifth character in `str`.

```
In[8]:= %[[5]]
Out[8]= s
```

`ToASCII` gives the numerical ASCII code for the character.

```
In[9]:= ToASCII[%]
Out[9]= 115
```

`FromASCII` converts from the code to the character.

```
In[10]:= FromASCII[%]
Out[10]= s
```

<code>StringJoin[str₁, str₂, ...]</code>	join several strings together
<code>StringLength[string]</code>	give the number of characters in a string
<code>str₁ == str₂</code>	test whether two strings are identical

Some functions for manipulating character strings.

`StringJoin` joins strings together.

```
In[11]:= StringJoin["The cat", " ", "in the hat"]
Out[11]= The cat in the hat
```

You can apply `StringJoin` to a list of characters.

```
In[12]:= Apply[StringJoin, {"a", "b", "c"}]
Out[12]= abc
```

This finds the total number of characters in `str`.
`In[13]:= StringLength[str]`
`Out[13]= 11`

You can use the standard `==` operator to find out if two strings are identical.
`In[14]:= "x y" == "x y"`
`Out[14]= True`

When you work with expressions in *Mathematica*, you are usually concerned with their *structure*, whether mathematical or otherwise. Sometimes, however, you may want to manipulate or use an expression in a purely textual way. You can do this by converting the expression to a string.

<code>ToString[expr]</code>	give the textual form of an expression as a string
<code>ToExpression[string]</code>	convert (if possible) a string to an expression

Converting between expressions and strings.

This gives a string of the `InputForm` text of the expression.
`In[15]:= ToString[x^2 + a/b]`
`Out[15]=  $\frac{a}{b} + x^2$`

This converts the string back to an expression.
`In[16]:= ToExpression[%]`
`Out[16]=  $2 a$`

You can use `ToString` and `ToExpression` to convert between names of symbols and strings. Sometimes, you may find it convenient to generate a sequence of symbols with different names. The *Mathematica* function `Unique` discussed in Section 2.7.2 allows you to do this.